

Cargoservice — Sprint2

ISS 2026 — VALERIA CAPACCI & GINEVRA PENTOLI

[GOAL](#)[PROBLEM ANALYSIS](#)[TEST PLANS](#)[PROJECT](#)[DEPLOYMENT](#)[SPRINT3](#)

Goal

Progettare e realizzare `cargorobot`, il componente che orchestra il movimento del DDR. Il suo compito è portare il container dall'IOPort a `slot5` (dove avviene la marcatura), e da lì allo slot riservato, per poi tornare in HOME (requisito **R3** del committente). Il goal comprende inoltre la separazione dei Context per nodo fisico.

Problem analysis

Cargorobot come componente intermedio

Come stabilito già in Sprint0, `RobotSmart26` è il servizio, fornito dal committente, che `cargorobot` utilizza come servizio esterno per il movimento del DDR, scelto fra le alternative disponibili (`RobotObj26`, `RobotService26`, `RobotSmart26`) perché è quella che riduce maggiormente l'abstraction gap: calcola autonomamente il percorso verso un punto di cui sono note le coordinate. `cargorobot` resta comunque un'entità applicativa da progettare e realizzare, distinta da `RobotSmart26`, con la responsabilità di orchestrare la sequenza di operazioni. È quindi un componente intermedio fra `RobotSmart26` e `cargoservice`.

`RobotSmart26` conosce infatti solo primitive di movimento verso coordinate e non ha alcuna nozione dei concetti di dominio del sistema (IOPort, slot5, marker, stiva, sequenza di carico) né dello stato di avanzamento di un'operazione di carico. Se `cargoservice` invocasse `RobotSmart26` direttamente dovrebbe gestire sia la logica di business dell'orchestrazione delle varie richieste di carico sia la comunicazione col servizio esterno `RobotSmart26`, con due conseguenze indesiderate: la logica di orchestrazione del DDR resterebbe sparsa nell'attore `cargoservice` invece di essere incapsulata in un'unica entità responsabile e un cambiamento nel servizio robot si propagherebbe fino a `cargoservice` invece di restare confinato al solo componente

che lo usa. Introdurre `cargorobot` come entità intermedia mantiene invece la stessa separazione delle responsabilità, andando a rispettare il Single Responsibility Principle.

Comunicare con RobotSmart26

Analizzando il servizio fornito RobotSmart26 notiamo che esistono diversi tipi di comunicazione possibile. Dobbiamo quindi valutare e scegliere, fra i canali che il servizio espone, quello che riteniamo più adatto per far comunicare `cargoservice` con il servizio:

Alternativa	Considerazioni
ExternalQActor	si dichiara un <code>ExternalQActor</code> che punta al Context su cui gira RobotSmart26. Da quel momento <code>cargorobot</code> può inviare <code>request / dispatch</code> verso <code>robotsmart</code> esattamente come farebbe con un attore locale: la serializzazione dei messaggi e il loro instradamento in rete sono gestiti automaticamente dall'infrastruttura qak.
MQTT	RobotSmart26 può essere raggiunto anche pubblicando sulla topic <code>robotsmart26in</code> di un broker; gli eventi/reply generati vengono ripubblicati sulla topic gemella <code>robotsmart26in_out</code> . Introduce quindi un componente architetturale aggiuntivo e soprattutto, a differenza del canale ExternalQActor, i payload vanno costruiti e correlati esplicitamente.
CoAP	RobotSmart26 espone anche un client CoAP dedicato, pensato però per l'osservazione delle risorse (lo stato di <code>robotmnemo</code>) piuttosto che per l'invio di comandi; usarlo per pilotare il robot richiederebbe di reimplementare da zero sia il mapping dei comandi sia la logica di correlazione fra richiesta e risposta, senza alcun beneficio rispetto alle altre due alternative nel contesto di cargoservice.

Si decide di adottare l' `ExternalQActor` nativo qak come canale primario fra `cargorobot` e RobotSmart26 perché non introduce componenti aggiuntivi (a differenza di MQTT) e non richiede logica di correlazione scritta a mano (a differenza di CoAP). Essendo RobotSmart26 esso stesso realizzato in qak, questo canale non richiede inoltre alcun livello di conversione dei messaggi.

I comandi di RobotSmart26

RobotSmart26 espone il proprio comportamento a più livelli di astrazione, non uno solo, e va quindi scelto quale usare per orchestrare il percorso che devono seguire i

container. I comandi rilevanti sono:

- `moverobot(TARGETX,TARGETY,STEPTIME) → moverobotok(ARG) / moverobotfailed(PLANDONE,PLANTODO)` : muove il robot verso una coppia di coordinate assolute, calcolando autonomamente il percorso e orientandosi verso l'obiettivo prima di partire;
- `buildPlan(PX,PY,TX,TY) → buildPlanDone(PLAN)` seguito da `doplan(PLAN,STEPTIME) → doplandone(ARG) / doplanfailed(PLANTODO)` : separa il calcolo del piano dalla sua esecuzione, lasciando al chiamante la possibilità di ispezionare o modificare il piano fra i due passi;
- `step(TIME) → stepdone(V) / stepfailed(DURATION,CAUSE)` e `move(M)` (dispatch, $M = |l| |r| |d| |h|$): comandi elementari di singolo passo/rotazione, pensati per scenari di esplorazione in cui la mappa non è nota a priori.

Analizzando le varie possibilità:

Alternativa	Considerazioni rispetto al caso d'uso
<code>moverobot</code>	massima riduzione dell'abstraction gap: <code>cargorobot</code> chiede solo "vai in (X,Y)", senza occuparsi di pianificazione. Adatta quando l'ambiente è noto a priori, come nel nostro caso.
<code>buildPlan</code> + <code>doplan</code>	utile se occorre ispezionare o adattare il piano prima di eseguirlo (ad esempio per evitare una zona); introduce però in <code>cargorobot</code> una responsabilità di orchestrazione del piano che, per le nostre tratte fisse, non ha controparte in alcun requisito.
<code>step</code> / <code>move</code>	livello più basso, pensato per l'esplorazione di un ambiente non ancora mappato; nel nostro caso l'ambiente è già interamente noto (le coordinate degli slot sono un dato di progetto), quindi usarlo significherebbe reimplementare in <code>cargorobot</code> una logica di path-planning che RobotSmart26 offre già.

Si sceglie `moverobot` come comando primario di movimento: le tratte che `cargorobot` deve percorrere sono verso coordinate fisse e note in anticipo, per cui non c'è alcun bisogno di intervenire sul piano calcolato.

Il marker

Il requisito descrive il marker come un dispositivo a sé, ma vale la pena chiedersi se avrebbe senso realizzarlo come una singola entità separata oppure se "inglobarlo" assieme a `cargorobot`. Analizzandone il comportamento notiamo infatti che esso si

occupa solo di gestire per ogni container che gli viene affidato un timeout (con cui notifica il DDR) e l'assegnazione dello slot. Avendo quindi minime responsabilità, ha pertanto senso gestirlo internamente all'orchestratore in modo "astratto", trattandolo come dettaglio implementativo.

Comportamento di cargorobot al guasto di RobotSmart26

Come stabilito precedentemente con il committente, in presenza di guasti il robot si ferma immediatamente e resta fermo finché il sistema non decide il da farsi.

`cargorobot` si limita quindi a riconoscere il fallimento e a fermarsi, mentre la decisione su come reagire resta a `cargoservice`.

Separazione dei Context

Precedentemente si era già individuata la necessità di alcuni nodi fisici distinti (`cargorobot`, `cargoservice`, `sonar`, `IOPort`), ma finora abbiamo mantenuto `cargoservice` e `ioport` in un unico Context per pura semplicità, rimandando la valutazione di un'eventuale separazione reale successivamente. Riconsiderando la questione, si decide di non introdurre un `ctxioport` dedicato. Il motivo è che la vera separazione fisica rilevante per `ioport` non è quella verso `cargoservice`, ma quella verso il browser che ospita la WebGUI e quest'ultima è già un processo separato per definizione (gira sul dispositivo del cliente, non su un nostro nodo), che comunica con l'attore `ioport` tramite `IOPortWsAdapter`. Introdurre un secondo processo qak solo per `ioport` aggiungerebbe quindi un salto di rete in più fra `ioport` e `cargoservice` (via `ExternalQActor`) senza portare alcun beneficio.

I nodi di questo sprint restano quindi:

- `ctxcargoservice` (`localhost:8050`): ospita gli attori `cargoservice`, `cargorobot` e `ioport`, oltre al POJO `Hold`.
- **RobotSmart26**: già esterno per definizione, referenziato come `ExternalQActor` nel Context di `cargoservice` su `localhost:8020`.

Il modello prodotto

Dall'analisi precedente segue il protocollo dei messaggi del modello `qak` (`sprint2.qak`).

[`sprint2.qak`](#)

```
Request loadrequest : loadrequest(OCCUPIED)
Reply  loadaccepted : loadaccepted(SLOT,HOLD) for loadrequest
```

```

Reply    loadrejected : loadrejected(REASON,HOLD)    for loadrequest

Dispatch pushButton    : pushButton(NONE)
Dispatch setOccupied    : setOccupied(FLAG)
Dispatch updateDisplay : updateDisplay(STATE,HOLD,MSG)

Request transportContainer : transportContainer(SLOT,TARGETX,TARGETY)
Reply    transportDone      : transportDone(SLOT)      for
transportContainer
Reply    transportFailed    : transportFailed(CAUSE)    for
transportContainer

Dispatch robotHalted : robotHalted(CAUSE)

Request moverobot      : moverobot(TARGETX, TARGETY, STEPTIME)
Reply    moverobotdone  : moverobotok(ARG)              for
moverobot
Reply    moverobotfailed: moverobotfailed(PLANDONE, PLANTODO)    for
moverobot

Context ctxcargoservice ip [host="localhost" port=8050]    //
cargoservice, cargorobot e ioport: tutti qui, un solo processo
Context ctxrobotsmart   ip [host="127.0.0.1" port=8020]    // unico
vero servizio esterno

ExternalQActor robotsmart context ctxrobotsmart

QActor cargoservice context ctxcargoservice
    withobj hold using "utils.cargoservice.Hold()" {

}

QActor cargorobot context ctxcargoservice
    withobj hold using "utils.cargoservice.Hold()" {

}

QActor ioport context ctxcargoservice
    withobj status using "utils.cargoservice.IOPortStatus()" {

}

```

Test plans

Test che guideranno l'implementazione di `cargorobot`.

Test	Obiettivo
Percorso completo del container	Verificare che <code>cargorobot</code> , ricevuto l'incarico di caricare un container, invii correttamente le richieste <code>moverobot</code> , rispettando l'ordine richiesto dal requisito R3 (IOPort → slot5 → slot riservato → HOME).
Fallimento del movimento	Simulare la risposta <code>moverobotfailed</code> e verificare che <code>cargorobot</code> si fermi immediatamente e resti in attesa, senza inviare ulteriori comandi finché il sistema non decide come procedere.
Comunicazione con RobotSmart26	Verificare che i messaggi <code>moverobot</code> / <code>moverobotok</code> / <code>moverobotfailed</code> scambiati fra <code>ctxcargoservice</code> e <code>ctxrobotsmart</code> tramite l' <code>ExternalQActor robotsmart</code> arrivino correttamente a destinazione, eseguendo i due Context come processi realmente separati.
Gestione degli slot	Verificare, tramite <code>Hold</code> , che richieste di carico successive vengano assegnate a slot liberi distinti (<code>reserveFirstFree</code>), che uno slot pieno/timeout venga correttamente liberato (<code>releaseReserved</code>) e che a stiva piena una nuova <code>loadrequest</code> venga rifiutata con motivo <code>full</code> .

Project

A partire dal modello di analisi si è realizzato un prototipo eseguibile che copre il goal dello sprint: orchestrazione del trasporto tramite `cargorobot`, gestione degli slot tramite `Hold`, mantenimento (senza modifiche strutturali) di `ioport`.

`cargorobot`

Realizzato come nuovo `QActor` nel Context `ctxcargoservice`. Ogni passo della sequenza invia una `moverobot` a `robotsmart` (`ExternalQActor`) e reagisce solo alla relativa reply; in `atSlot5` l'attesa della marcatura è gestita con un timeout fisso di 5 s (`whenTime 5000`):

[sprint2.qak](#)

```
QActor cargorobot context ctxcargoservice
  withobj hold using "utils.cargoservice.Hold()" {
```

```
[# var TargetSlot=-1, TargetX=-1, TargetY=-1 #]

State idle initial { println("$name | idle") }
Transition t0 whenRequest transportContainer -> startTransport

State startTransport {
    onMsg(transportContainer :
transportContainer(SLOT,TARGETX,TARGETY)) {
        [# TargetSlot=payloadArg(0).toInt()
        TargetX=payloadArg(1).toInt()
        TargetY=payloadArg(2).toInt()
        var IOPort=hold.getIOPortPosition()
        var X_IOPort=IOPort.getX(), Y_IOPort=IOPort.getY() #]
        request robotsmart -m moverobot :
moverobot($X_IOPort,$Y_IOPort,345)
    }
}
Transition t1
    whenReply moverobotdone -> atIOPort
    whenReply moverobotfailed -> halted

State atIOPort { }
Transition t2 whenTime 2000 -> goToSlot5

State goToSlot5 {
    [# var S5=hold.getSlot5Position()
    var X_S5=S5.getX(), Y_S5=S5.getY() #]
    request robotsmart -m moverobot : moverobot($X_S5,$Y_S5,345)
}
Transition t3
    whenReply moverobotdone -> atSlot5
    whenReply moverobotfailed -> halted

State atSlot5 { }
Transition t4 whenTime 2000 -> goToReservedSlot

State goToReservedSlot {
    request robotsmart -m moverobot :
moverobot($TargetX,$TargetY,345)
}
Transition t5
    whenReply moverobotdone -> atSlot
    whenReply moverobotfailed -> halted

State atSlot { }
Transition t6 whenTime 2000 -> goHome

State goHome {
```

```

        [# var H=hold.getHomePosition()
          var X_HOME=H.getX(), Y_HOME=H.getY() #]
        request robotsmart -m moverobot :
moverobot($X_HOME,$Y_HOME,345)
    }
    Transition t7
        whenReply moverobotdone -> done
        whenReply moverobotfailed -> halted

    State done {
        replyTo transportContainer with transportDone :
transportDone($TargetSlot)
        [# TargetSlot=-1, TargetX=-1, TargetY=-1 #]
    }
    Goto idle

    State halted {
        forward cargoservice -m robotHalted :
robotHalted(guastoRobotSmart26)
        replyTo transportContainer with transportFailed :
transportFailed(guastoRobotSmart26)
    }
}

```

cargoservice

engaged incarica **cargorobot** passandogli anche la posizione dello slot riservato, e la transizione da **engaged** osserva sia il timeout dei 30s sia le reply **transportDone** / **transportFailed** di **cargorobot**: un fallimento riporta subito a **disengaged** (stesso comportamento del timeout), mentre un trasporto concluso con successo porta a un nuovo stato **success** dedicato, che aggiorna la stiva e il display con un messaggio diverso da quello di timeout:

[sprint2.qak](#)

```

QActor cargoservice context ctxcargoservice
withobj hold using "utils.cargoservice.Hold()" {

    [# var Slot=-1
      var Occupied=false
      hold.setAllSlotsFree() #]

    State idle initial { }
    Transition t0 whenRequest loadrequest -> handleRequest

    State handleRequest {

```

```

onMsg(loadrequest : loadrequest(X)) {
  [# Occupied=payloadArg(0).toBoolean()
   var H="" + hold.displayStatus() + "" #]
  if [# Occupied == true #] {
    replyTo loadrequest with loadrejected :
loadrejected(retrylater,$H)
  } else {
    [# Slot=hold.reserveFirstFree() #]
    if [# Slot != -1 #] {
      [# var H2="" + hold.displayStatus() + "" #]
      replyTo loadrequest with loadaccepted :
loadaccepted($Slot,$H2)
    } else {
      replyTo loadrequest with loadrejected :
loadrejected(full,$H)
    }
  }
}

Goto engaged if [# Slot != -1 && Occupied == false #] else idle

State engaged {
  [# var TX=hold.slotX(Slot), TY=hold.slotY(Slot) #]
  request cargorobot -m transportContainer :
transportContainer($Slot,$TX,$TY)
}
Goto waitingTransport

State waitingTransport { }
Transition t1 whenTime 30000      -> disengaged
      whenReply transportDone      -> success
      whenReply transportFailed    -> disengaged
      whenRequest loadrequest      -> rejectBusy

State rejectBusy {
  onMsg(loadrequest : loadrequest(X)) {
    [# var H="" + hold.displayStatus() + "" #]
    replyTo loadrequest with loadrejected :
loadrejected(retrylater,$H)
  }
}
Goto waitingTransport

State disengaged {
  [# hold.releaseReserved()
   var H="" + hold.displayStatus() + ""
   var S="'IDLE'"
   var M="'Timeout scaduto: slot liberato, turno idle'" #]

```

```

        forward ioport -m updateDisplay : updateDisplay($S,$H,$M)
        [# Slot=-1 #]
    }
    Goto idle

    State success {
        onMsg(transportDone : transportDone(SLOT)) {
            [# Slot=payloadArg(0).toInt()
            var Done=hold.setReservedSlotOccupied()
            hold.releaseReserved()
            var H="" + hold.displayStatus() + ""
            var S=""IDLE""
            var M=""Trasporto terminato"" #]
            forward ioport -m updateDisplay : updateDisplay($S,$H,$M)
            [# Slot=-1 #]
        }
    }
    Goto idle
}

```

Gestione degli slot

La gestione della stiva è incapsulata nel POJO `Hold` che mantiene due tipi di informazione ben distinti:

- **layout fisso** della stiva (posizioni di IOPort, HOME, slot5 e dei 4 slot), definito come costante nel costruttore di `Hold` e mai modificato: può quindi essere letto in sicurezza da più istanze di `Hold` senza alcuno stato condiviso;
- **quale slot è occupato**, essendo mutabile vive solo nell'istanza di `Hold` di `cargoservice`.

Questa distinzione permette a `cargorobot` di istanziare una propria copia di `Hold` in sola lettura, usata solo per leggere le posizioni fisse (slot5, HOME, IOPort) tramite `getSlot5Position()` / `getHomePosition()` / `getIOPortPosition()`, senza toccare mai lo stato delle riserve.

I metodi di `Hold` che `cargoservice` usa sono:

Metodo	Uso
<code>reserveFirstFree()</code>	assegna il primo slot libero e lo marca occupato, oppure ritorna <code>-1</code> se la stiva è piena

<code>releaseReserved()</code>	chiamato in <code>disengaged</code> (timeout scaduto senza che il container sia mai entrato)
<code>slotX(id) / slotY(id)</code>	chiamati in <code>engaged</code> per calcolare le coordinate da passare a <code>cargorobot</code> nella <code>transportContainer</code>
<code>displayStatus()</code>	usato in ogni risposta/notifica verso <code>ioport</code> , per mostrare lo stato della stiva sul display

[Hold.java](#)

```

public int reserveFirstFree() {
    for (Pair<IPosition, ISlot> p : slotList) {
        ISlot slot = p.getSecond();
        if (!slot.isOccupied()) {
            slot.setOccupied(true);
            reservedSlotId = slot.getID();
            return reservedSlotId;
        }
    }
    return -1; // stiva piena
}

public void releaseReserved() {
    if (reservedSlotId == -1) return;
    findById(reservedSlotId).getSecond().setOccupied(false);
    reservedSlotId = -1;
}

public String displayStatus() {
    StringBuilder sb = new StringBuilder();
    for (Pair<IPosition, ISlot> p : slotList) {
        ISlot slot = p.getSecond();
        if (sb.length() > 0) sb.append(",");
        sb.append(slot.getID()).append(":").append(slot.isOccupied() ?
"OCC" : "FREE");
    }
    return sb.toString();
}

```

Deployment

Il sistema è composto da due processi separati: il nodo applicativo

(`ctxcargoservice`) e il servizio esterno RobotSmart26 (`ctxrobotsmart`), oltre al

browser che ospita la WebGUI, già di per sé un processo indipendente.

Nodo	Indirizzo	Contenuto
ctxcargoservice	localhost:8050	attori <code>cargoservice</code> , <code>cargorobot</code> , <code>ioport</code> ; classi <code>utils.cargoservice.*</code> (<code>Hold</code> , <code>Slot</code> , <code>Position</code> , ...)
ctxrobotsmart	127.0.0.1:8020	RobotSmart26, fornito dal committente, avviato separatamente tramite <code>robotsmart26.yaml</code> / Docker
WebGUI	ws://localhost:7070	pagina browser del cliente, connessa via <code>IOPortWsAdapter</code>

Definizione dello Sprint3

REQUISITI

- Progettare e realizzare l'attore `sonar` e scrivere il firmware del microcontrollore Raspberry che misura la distanza `D` davanti all'IOPort.
- Gestire la reazione del sistema al guasto sonar durante il movimento del DDR.
- Completare la gestione del `LED`.



Valeria Capacci

valeria.capacci@studio.unibo.it



Ginevra Pentoli

ginevra.pentoli@studio.unibo.it

GIT: github.com/valecapa/TemaFinaleISS2026